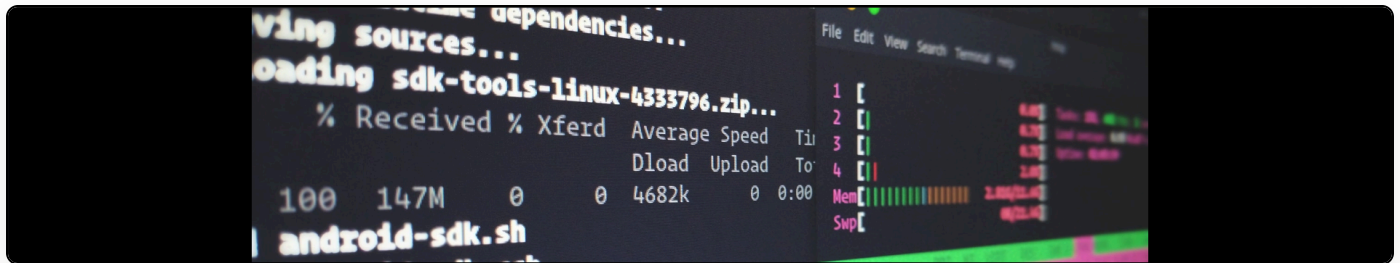


Debugging meTypeset using a git filesystem

Cite as: Martin Paul Eve, "Debugging meTypeset using a git filesystem", *eve.gd: Martin Paul Eve*, February 27, 2014, <https://doi.org/10.59348/5vnbp-c9c33>.



Debugging a text-based transcoder

meTypeset is, in essence, a transcoder for text. While “transcode” is usually used in a multimedia context, we are transcoding from one XML specification (Microsoft's OOXML) to another (JATS XML). This involves several stages of action:

- Unzip the document
- Perform XSLT transforms to an intermediate format
- Do some logic-based guesswork on what the author might have meant with their strange formatting
- Transform to NLM/JATS

There is potential for unexpected results at every stage of this process.

Enter git debug filesystem

While it is possible, when developing, to step through most of the processes, because we have multiple portions of the transform handled by different technologies, it is often difficult to pinpoint the stage at which something went wrong. For instance: if the NLM isn't right, was the TEI right? If the TEI isn't right, was it right before we put it through python (and which module messed it up?)

To solve this, when meTypeset is passed the debug flag (“-d” or “--debug”) it will now initialize all of its output directories as git repositories and regularly commit after each module has performed its transforms, thereby providing an easy way of logging in any environment (and cloning the output to another machine). As a self-contained filesystem, git is ideal for this kind of work. It adds very little overhead (either in terms of space or processing time) and makes life a lot easier in this kind of debug work. You can see the implementation of this in GitPython in the dev branch of the project.

Cite this article

Please include the DOI in your citation: <http://dx.doi.org/10.6084/m9.figshare.946260>

You can [view this post's XML with lens](#).